TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TOÀN DIỆN PHẦN 3: LỚP MẢNG ĐỐI TƯỢNG, FILE TEXT & STATIC

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (OOP JAVA) • PHẦN 3

Phân công: Thành viên 3 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 3

> **Mục tiêu:** Nắm vững cấu trúc Lớp mảng danh sách đối tượng (Class Array / Collection Wrapper), cơ chế xử lý file text chuẩn hóa UTF-8 chống lỗi font tiếng Việt, cơ chế cộng dồn giỏ hàng và bản chất các thành phần Static trong Java.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 3

Phần 3 gồm tổng cộng **4 tập tin mã nguồn (.java) và 3 tập tin dữ liệu (.txt)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 4 Lớp

1. `DanhSachSanPham.java` : Lớp mảng chuyên biệt quản lý tập hợp toàn bộ sản phẩm trong kho hàng.
2. `DanhSachKhachHang.java` : Lớp mảng chuyên biệt quản lý danh sách người dùng (Khách hàng VIP, Thân thiết, Nhân viên, Quản lý).
3. `ChiTietHoaDon.java` : Lớp biểu diễn một dòng sản phẩm mua trong hóa đơn (gồm sản phẩm và số lượng).
4. `HoaDon.java` : Lớp quản lý đơn hàng, tính tiền đa hình, in hóa đơn và in phiếu bảo hành.

1.2. Các tập tin dữ liệu lưu trữ (Data Files) - 3 File

5. `products.txt` : File văn bản lưu trữ thông tin 6 loại sản phẩm thời trang.
6. `users.txt` : File văn bản lưu trữ thông tin khách hàng và nhân sự cửa hàng.
7. `hoadon.txt` : File văn bản lưu vết lịch sử mọi hóa đơn thanh toán và phiếu bảo hành đã xuất.

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Lớp mảng các đối tượng (Class Array)**	`DanhSachSanPham` , `DanhSachKhachHang`	Đóng gói mảng `ArrayList` bên trong class; không để lộ cấu trúc lưu trữ; cung cấp đầy đủ hàm CRUD (Xem, Thêm, Sửa, Xóa, Tìm kiếm).
Kỹ thuật chính	**Quan hệ Chứa đựng (Aggregation)**	`HoaDon` chứa `ArrayList<ChiTietHoaDon>` , `ChiTietHoaDon` chứa `SanPhamThoiTrang`	Mô hình hóa mối quan hệ thực tế: Một hóa đơn gồm nhiều món hàng, mỗi món

			gồm mặt hàng và số lượng tương ứng.
Kỹ thuật chính	**Thành phần Static (Tĩnh)**	`SanPhamThoiTrang.TI_LE_VAT`, `HoaDon.tongSoHoaDon`, `formatTien()`	Biến static lưu trữ vùng nhớ duy nhất mức lớp; phương thức static tiện ích dùng chung không cần tạo đối tượng.
Kỹ thuật chính	**Đọc/Ghi luồng File Text**	`docTuFile()`, `ghiVaoFile()`, `xuatHoaDonRaFile()`	Lưu trữ trạng thái bền vững xuống đĩa cứng, tự nạp lại dữ liệu khi khởi động lại chương trình.
Kỹ thuật phụ trợ	**Chuẩn hóa bảng mã UTF-8**	`StandardCharsets.UTF_8` kết hợp `InputStreamReader/OutputStreamWriter`	Chống 100% lỗi font tiếng Việt có dấu ("Áo gió ấm áp", "Váy dạ hội") trên mọi hệ điều hành Windows/Linux.
Kỹ thuật phụ trợ	**Cấu trúc Try-with-resources**	`try (BufferedReader reader = ...) { ... }`	Tự động đóng luồng (`reader.close()`) ngay khi đọc xong hoặc có lỗi ngoại lệ, chống rò rỉ bộ nhớ.
Kỹ thuật phụ trợ	**Thuật toán Tự sinh mã ID**	`taoMaTuDong(type)` trong `DanhSachSanPham.java`	Quét mã lớn nhất hiện có và tự động tăng dần (`AK001`, `AK002` ...), người dùng không cần gõ mã thủ công.
Kỹ thuật phụ trợ	**Cộng dồn số lượng giỏ hàng**	`tangSoLuong(int amount)` trong `ChiTietHoaDon.java`	Khi mua lặp lại cùng một mã hàng, hệ thống tự động tăng số lượng thay vì tạo thêm dòng mới.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Lớp mảng danh sách sản phẩm `DanhSachSanPham.java`

Đoạn Code A: Đóng gói tập hợp `ArrayList`

```
public class DanhSachSanPham {
    private ArrayList<SanPhamThoiTrang> danhSach;

    public DanhSachSanPham() {
        this.danhSach = new ArrayList<>();
    }

    public ArrayList<SanPhamThoiTrang> getDanhSach() { return danhSach; }
    public int getSoLuong() { return danhSach.size(); }
    public boolean isEmpty() { return danhSach.isEmpty(); }
```

- **Kỹ thuật chính:** Tính Đóng gói (Encapsulation). Thuộc tính `danhSach` được đặt `private`. Mọi thao tác truy cập số lượng hay duyệt danh sách đều phải thông qua các phương thức của lớp mảng.

Đoạn Code B: Thuật toán Tự sinh mã ID tự động (Kỹ thuật phụ trợ nâng cao)

```
public String taoMaTuDong(String type) {
        String prefix = "SP";
        if (type != null) {
            String t = type.toLowerCase();
            if (t.equals("aokhoac")) prefix = "AK";
            else if (t.equals("quanjean") || t.equals("jeans")) prefix = "QJ";
            else if (t.equals("aothun") || t.equals("tshirt")) prefix = "AT";
            else if (t.equals("phukien") || t.equals("accessory")) prefix = "PK";
            else if (t.equals("vaydam")) prefix = "VD";
            else if (t.equals("giaydep")) prefix = "GD";
        }

        int maxId = 0;
        for (SanPhamThoiTrang p : danhSach) {
            if (p.getMaSP() != null && p.getMaSP().toUpperCase().startsWith(prefix)) {
                String numPart = p.getMaSP().substring(prefix.length());
                try {
                    int id = Integer.parseInt(numPart);
                    if (id > maxId) maxId = id;
                } catch (NumberFormatException ignored) {}
            }
        }
        return String.format("%s%03d", prefix, maxId + 1);
    }
```

- **Giải thích từng dòng:**
- `prefix`: Xác định tiền tố theo loại (`AK`, `AT`, `QJ`, `PK`, `VD`, `GD`).
- Vòng lặp `for`: Quét qua toàn bộ sản phẩm trong danh sách.
- `p.getMaSP().substring(prefix.length())`: Tách bỏ tiền tố để lấy phần số. Ví dụ `"AK007"` cắt bỏ 2 ký tự đầu lấy được chuỗi `"007"`.
- `Integer.parseInt(numPart)`: Chuyển chuỗi thành số nguyên và tìm số lớn nhất `maxId`.
- `String.format("%s%03d", prefix, maxId + 1)`: Định dạng mã mới với 3 chữ số đệm (ví dụ `maxId` là 7 thì sinh `AK008`).

Đoạn Code C: Nghiệp vụ CRUD đầy đủ

```
public boolean them(SanPhamThoiTrang sp) {
        if (sp == null || sp.getMaSP() == null || sp.getMaSP().trim().isEmpty()) return false;
        if (timKiem(sp.getMaSP()) != null) return false;
        danhSach.add(sp);
        return true;
    }

    public boolean xoa(String maSP) {
        if (maSP == null) return false;
        for (int i = 0; i < danhSach.size(); i++) {
            if (danhSach.get(i).getMaSP() != null &&
danhSach.get(i).getMaSP().equalsIgnoreCase(maSP.trim())) {
                danhSach.remove(i);
                return true;
            }
        }
        return false;
```

```
    }

    public boolean sua(String maSP, String tenMoi, double giaMoi) {
        SanPhamThoiTrang sp = timKiem(maSP);
        if (sp == null) return false;
        if (tenMoi != null && !tenMoi.trim().isEmpty()) sp.setTenSP(tenMoi.trim());
        if (giaMoi >= 0) sp.setGiaBan(giaMoi);
        return true;
    }

    public SanPhamThoiTrang timKiem(String maSP) {
        if (maSP == null) return null;
        for (SanPhamThoiTrang sp : danhSach) {
            if (sp.getMaSP() != null && sp.getMaSP().equalsIgnoreCase(maSP.trim())) return sp;
        }
        return null;
    }
```

- **Kỹ thuật chính:** Đảm bảo đầy đủ 4 thao tác chuẩn CRUD:
- **Create (Thêm):** Có kiểm tra trùng mã sản phẩm.
- **Read/Search (Tìm kiếm):** Tìm theo mã không phân biệt hoa thường.
- **Update (Sửa):** Tìm và cập nhật tên, đơn giá.
- **Delete (Xóa):** Tìm vị trí phần tử và gọi `remove(i)` .

3.2. Kỹ thuật Đọc/Ghi File Text chuẩn UTF-8

```
public void docTuFile(String fileName) {
        File file = new File(fileName);
        if (!file.exists()) return;

        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] parts = line.split(",");
                if (parts.length < 5) continue;

                String type = parts[0].trim();
                String code = parts[1].trim();
                String name = parts[2].trim();
                double price = Double.parseDouble(parts[3].trim());

                switch (type.toLowerCase()) {
                    case "aokhoac":
                        boolean haveHat = Boolean.parseBoolean(parts[4].trim());
                        them(new AoKhoac("AoKhoac", code, name, price, haveHat));
                        break;
                    case "giaydep":
                        int szGiay = Integer.parseInt(parts[4].trim());
                        them(new GiayDep("GiayDep", code, name, price, szGiay));
                        break;
                    // ... các sản phẩm khác
                }
            }
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi doc du lieu tu file.");
        }
    }
```

```
public void ghiVaoFile(String fileName) {
        try (BufferedWriter writer = new BufferedWriter(
                new OutputStreamWriter(new FileOutputStream(fileName), StandardCharsets.UTF_8))) {
            for (SanPhamThoiTrang sp : danhSach) {
                writer.write(sp.getStringSaveFile());
                writer.newLine();
            }
            writer.flush();
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi luu du lieu vao file.");
        }
    }
```

- **Kỹ thuật chính:**
- `sp.getStringSaveFile()` : Gọi đa hình hàm tạo chuỗi của từng đối tượng để ghi file.
- **Kỹ thuật phụ trợ:**
- `Try-with-resources` : Tự động giải phóng tài nguyên luồng đọc/ghi.
- `StandardCharsets.UTF_8` : Bắt buộc mã hóa UTF-8 bảo toàn tiếng Việt có dấu.
- Khối `try-catch (NumberFormatException)` khi đọc đơn giá: Nếu có dòng bị lỗi số thì chương trình bỏ qua dòng đó chứ không bị crash đột ngột.

3.3. Quan hệ Chứa đựng trong `ChiTietHoaDon` và `HoaDon`

```
public class ChiTietHoaDon {
    private SanPhamThoiTrang sanPham;
    private int soLuong;

    public void tangSoLuong(int amount) {
        if (amount > 0) this.soLuong += amount;
    }

    public double tinhThanhTienSauGiam(double phanTramGiamGia) {
        if (sanPham == null) return 0;
        double giaSauGiam1SP = sanPham.calculateAfterDiscount(phanTramGiamGia);
        return (giaSauGiam1SP + sanPham.VATcalculation()) * soLuong;
    }
}
```

- **Kỹ thuật chính (Aggregation):** Mỗi dòng hóa đơn giữ 1 con trỏ đến đối tượng sản phẩm và biến `soLuong` .
- **Kỹ thuật phụ trợ:** Hàm `tangSoLuong` : Khi khách hàng thêm món hàng đã có trong hóa đơn, chương trình cộng dồn số lượng chứ không thêm phần tử trùng vào danh sách.

3.4. Các thành phần Static trong hệ thống

```
public static final double TI_LE_VAT = 0.10;
private static int tongSoLuongSanPhamDaTao = 0;
private static int tongSoHoaDon = 0;
```

- **Kỹ thuật chính:**
- `TI_LE_VAT` : Hằng số tĩnh đại diện 10% thuế VAT.
- `tongSoLuongSanPhamDaTao` : Tăng lên khi `new SanPhamThoiTrang` .

- `tongSoHoaDon` : Biến static trong lớp `HoaDon` , tự động tăng khi hoàn tất thanh toán 1 hóa đơn mới, phục vụ việc thống kê doanh số.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Lớp mảng các đối tượng là gì? Trong đồ án nhóm em cài đặt lớp mảng ở đâu và tại sao không dùng mảng tĩnh `SanPhamThoiTrang[]`?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Lớp mảng (Class Array / Collection Wrapper) là một lớp hướng đối tượng đóng gói một mảng hoặc danh sách bên trong, đồng thời cung cấp các phương thức nghiệp vụ tương ứng như Thêm, Sửa, Xóa, Tìm kiếm, Đọc/Ghi file. Trong đồ án, nhóm em cài đặt 2 lớp mảng chuyên biệt là `DanhSachSanPham` và `DanhSachKhachHang`.

2. Nhóm không dùng mảng tĩnh nguyên thủy `SanPhamThoiTrang[]` vì mảng tĩnh có kích thước cố định, khi thêm sản phẩm vượt quá số lượng sẽ bị lỗi tràn mảng hoặc phải viết hàm cấp phát lại bộ nhớ thủ công rất tốn kém. Việc đóng gói `ArrayList` trong lớp mảng vừa tận dụng được khả năng tự co giãn kích thước linh hoạt, vừa đảm bảo tính đóng gói của OOP."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `new InputStreamReader(..., StandardCharsets.UTF_8)` và hỏi: "Tại sao phải chỉ định `StandardCharsets.UTF_8`? Nếu bỏ đi có chạy được không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Nếu bỏ `StandardCharsets.UTF_8` và chỉ dùng `new FileReader(file)`, Java sẽ lấy bảng mã mặc định của hệ điều hành máy tính đang chạy (trên Windows thường là Windows-1258 hoặc Windows-1252). Khi đó, các chuỗi tiếng Việt có dấu như 'Áo gió thể thao', 'Giày dép Biti's' sẽ bị lỗi font, biến thành ký tự lạ hoặc dấu hỏi chấm (`??`).

2. Việc chỉ định tường minh `StandardCharsets.UTF_8` ép buộc máy ảo Java đọc đúng định dạng Unicode 8-bit quốc tế, đảm bảo chương trình chạy đồng nhất và hiển thị tiếng Việt hoàn hảo trên mọi hệ điều hành (Windows, Linux, macOS)."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Cú pháp `try (BufferedReader reader = ...)` khác gì với khối `try-catch` bình thường?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là cấu trúc **Try-with-resources** được giới thiệu từ Java 7:

1. Ở `try-catch` truyền thống, lập trình viên phải tự viết lệnh `reader.close()` trong khối `finally` và phải bọc thêm một `try-catch` nữa rất rườm rà. Nếu quên đóng, file sẽ bị khóa (file lock) và gây rò rỉ tài nguyên hệ thống.

2. Ở `Try-with-resources`, bất kỳ luồng nào khai báo trong ngoặc đơn của `try` sẽ được Java **tự động gọi hàm `close()` ngay khi khối lệnh kết thúc**, kể cả khi xảy ra Exception. Cách viết này vừa ngắn gọn, vừa an toàn tuyệt đối cho hệ thống."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Phân biệt thuộc tính Static và thuộc tính thông thường? Biến `tongSoHoaDon` hoạt động thế nào?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Thuộc tính thông thường thuộc về từng đối tượng cụ thể (Instance variable). Mỗi lần gọi `new`, một bản sao vùng nhớ mới được cấp phát riêng.

2. Thuộc tính Static thuộc về mức Lớp (Class-level). Toàn bộ chương trình chỉ có duy nhất một bản sao vùng nhớ

dùng chung cho tất cả các đối tượng.
3. Biến `HoaDon.tongSoHoaDon` là static, nên dù trong phiên làm việc chúng em tạo 100 hóa đơn khác nhau, biến này vẫn được chia sẻ chung và tự động tăng dần để đếm chính xác tổng số đơn hàng đã phục vụ."